
Stored Procedures

PROF. ME. JEFFERSON PASSERINI



Programação em SGDB's

Functions, Procedures e Triggers

Permite a programação de códigos em um SGBD.

Por que programar no SGBD?

Linguagens:

- **plpgSQL**, Tcl, Perl, Python e C.

Vantagens da plpgSQL:

- Utilização de comandos, tipos de dados e funções SQL em sua sintaxe.

Stored procedures e Functions



Stored Procedures

Quando uma aplicação solicita a execução de uma *query* SQL comum, todo o texto da mesma é enviado pela rede ao servidor onde será normalmente compilado e executado. O que pode gerar uma certa demora na resposta da *query*.

Procedimentos Armazenados podem ser entendidos como uma sequência de comandos SQL's agrupados, que são executados de forma sequencial.

No PostgreSQL não existe Procedimentos, mas sim Funções. Quando estivermos falando de Procedimentos, devemos entender Funções.

Vantagens e Desvantagens

Vantagens:

- Podem ser criadas rotinas especializadas altamente reutilizáveis, o que torna extremamente produtivo em ambientes do tipo *cliente/servidor*.
- As rotinas rodam no servidor, liberando esta carga do cliente.
- Redução da circulação de dados na rede, já que as *procedures* são executadas pelo SGBD no servidor;
- Pode ser acessada por várias aplicações;
- Segurança do banco de dados uma vez que elas podem acessar tabelas que o usuário não tem o direito de fazê-lo.

Desvantagem:

- Ficar restrito a sintaxe de um SGBD específico.

Funções

Blocos de códigos SQL armazenados no servidor de um BD.

Podem ser chamadas a qualquer momento com o objetivo de realizar algum processamento de informações.

São comparadas as Stored Procedures, porém são diferentes.



Functions X Stored Procedures

Funções: Realizar pequenas operações, normalmente auxiliares, estas podem ser solicitadas por uma transação.

São responsáveis pelo tratamento de textos, variáveis, formatação, operações repetitivas e rotineiras e podem ser utilizadas pelas Stored Procedure, já que esta não possui retorno de valores ou parâmetros.

Stored Procedures: Realizar transações completas e são encarregadas pela lógica do negócio (como um todo).



Functions X Stored Procedures

Concluindo:

- Funções podem e devem ser mescladas com Stored Procedures, porém esta, deve gerenciar as transações, já as funções disponibilizar recursos de bibliotecas de códigos utilizados frequentemente.

A sintaxe para a criação das duas é semelhante no PostgreSQL.

Sintaxe

Criação de uma Function:

- CREATE FUNCTION nome(argumentos) RETURNS tipo_de_retorno AS \$\$

OU

- CREATE **OR REPLACE** FUNCTION nome(argumentos) RETURNS tipo_de_retorno AS \$\$

Declarando a Linguagem da Function:

- \$\$ language plpgsql;

Sintaxe

Terminador de Comandos (Importante)

- No PostgreSQL, as funções devem ficar entre duplo cifrão \$\$

Comentários

- - - <comentários>

Begin...End

- Encapsula uma sequência de instruções, transformando-as em um bloco único de execução. **Pode ou não existe vírgula após o END.**

Begin

<Comandos>

End

Sintaxe

Parâmetros e Variáveis

Um procedimento armazenado pode receber argumentos (parâmetros) e retornar valores. Valores são retornados com o comando **RETURNS**.

- *Returns <nome do parâmetro> <tipo de dado>*

Para declarar variáveis locais, utilizamos a instrução **DECLARE VARIABLE**.

- *Declare Variable <nome da variável> <tipo de dado>;*

Sintaxe Completa

Sintaxe:

```
CREATE FUNCTION nome(argumentos) RETURNS tipo_de_retorno AS $$
```

```
DECLARE
```

```
--bloco de declarações das variáveis que serão utilizadas
```

```
BEGIN
```

```
--corpo da função
```

```
[RETURN variável;]
```

```
END;
```

```
$$ language plpgsql;
```

Sintaxe Completa

Referência para linguagem PL/pgSQL

<https://www.postgresql.org/docs/8.1/plpgsql.html>

Executando a Function

Para executar uma função basta invocá-la em um SELECT com o uso correto de seus parâmetros:

Sintaxe:

- `SELECT nomedafuncao (parâmetro 1, parâmetro 2);`

Criar Banco Exemplo

Criar um banco de dados como nome de **bdfatura**

Executar o scripts fornecido na aula.

Exemplo 1

Selecionando Dados

Faça uma função que retorne a frase: Ola Mundo!!

create or replace function sp_ola() returns text as

\$\$

begin

return ' Ola Mundo!!' ;

end;

\$\$

language PLPGSQL;

Utilizando a Função: `select sp_ola();`

Exemplo 2

Faça uma função que retorne uma frase passada como parâmetro de entrada.

create or replace function sp_ola2(frase text) returns text as

\$\$

begin

return frase;

end;

\$\$

language PLPGSQL;

Utilizando a Função: select sp_ola2('Ola Mundo!!');

Exemplo 3

Criar uma função que realiza a soma de dois parâmetros recebidos pela mesma.

```
CREATE FUNCTION soma(valorA integer, valorB integer) RETURNS integer AS $$
```

```
DECLARE
```

```
valor_total integer:=0;
```

```
BEGIN
```

```
valor_total = valorA + valorB;
```

```
RETURN valor_total;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
Select soma(5,3);
```



Exemplo 4

Criar uma função que realiza a soma das vendas

```
create or replace function valor_vendas() returns decimal(15,2) as
$$
declare valorvenda decimal(15,2);
begin
    select into valorvenda sum(valoritem) from itemvendas;
    return valorvenda;
end;
$$
language PLPGSQL;
```

Exemplo 4

UTILIZANDO SOBREPOSIÇÃO DE FUNÇÕES NO POSTGRESQL

```
create or replace function valor_vendas(id_venda int) returns decimal(15,2) as
$$
declare valorvenda decimal(15,2);
begin
    if (id_venda > 0) then
        select into valorvenda sum(valoritem) from itemvendas where
            idvenda=id_venda;

    else
        return 0;
    end if;
    return valorvenda;
end;
$$
language PLPGSQL;
```

Exemplo 5

Pegando um registro de um SQL e verificando o retorno.

```
create or replace function gera_entrega_venda(id_venda int) returns text as  
$$  
DECLARE  
    vendas_rec RECORD;  
BEGIN  
    select into vendas_rec * from venda where idvenda = id_venda;  
  
    if vendas_rec.enderecoentrega IS NULL or vendas_rec.enderecoentrega = '' THEN  
        RETURN 'Endereço não informado';  
    else  
        return vendas_rec.enderecoentrega;  
    END IF;  
END;$$  
language PLPGSQL;
```

Exemplo 6

Retornando uma informação tipo RECORD

create or replace function lista_venda()

returns setof record

as

\$\$

declare reg record;

begin

for reg in

(select i.idvenda, i.idproduto, p.descproduto, v.idcliente, c.nomecliente,
i.quantidade

from itemvendas as i

inner join venda as v on v.idvenda = i.idvenda

inner join produto as p on p.idproduto = i.idproduto

inner join cliente as c on c.idcliente = v.idcliente)

loop

return next reg;

end loop;

return;

end;

\$\$ language plpgsql;

Exemplo 6

Retornando uma informação tipo RECORD – utilizando a função

```
select * from lista_venda() as (idvenda int, idproduto int, descproduto varchar,  
idcliente int, nomecliente varchar, quantidade numeric);
```

Exemplo 7

Inserindo informações em outra tabela

1-Criar nova tabela

```
create table teste (  
    idvenda int,  
    idproduto int,  
    descproduto varchar(100),  
    idcliente int,  
    nomecliente varchar(100),  
    quantidade numeric(15,3));
```

Exemplo 7

Inserindo informações em outra tabela

2-Criar a função

```
create or replace function lista_venda2()  
returns setof record  
as  
$$  
begin  
    delete from teste;  
    insert into teste(idvenda,idproduto,descproduto,idcliente,nomecliente,quantidade)  
(select i.idvenda, i.idproduto, p.descproduto, v.idcliente, c.nomecliente, i.quantidade  
    from itemvendas as i  
        inner join venda as v on v.idvenda = i.idvenda  
        inner join produto as p on p.idproduto = i.idproduto  
        inner join cliente as c on c.idcliente = v.idcliente);  
  
    return;  
end;  
$$ language plpgsql;
```

Select lista_venda2();

Alterando Funções

Para alterar qualquer item de uma função basta recria-la com o comando CREATE OR REPLACE FUNCTION.

Pode-se alterar também:

Somente seu Nome:

- ALTER FUNCTION *nome_antigo*([argumentos]) RENAME TO *novo_nome*;

Ou Proprietário:

- ALTER FUNCTION *nome*([argumentos]) OWNER TO *novo_usuario*;

Excluindo Funções

Sintaxe:

DROP FUNCTION nome([argumentos]) opções;

Onde as opções podem ser:

- CASCADE: excluirá todos os objetos diretamente dependentes da mesma.
- RESTRICT: somente a função será removida.